

COMP101

Introduction to Programming

2025-26

Tutorial-03
Languages, Algorithms
Problem Solving

Languages

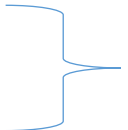
- Program languages have the same basic functionality
 - Constructs (sequence, selection, iteration) data types, operations
- Languages are classed via ‘paradigms’
 - The way we program varies from language to language
 - COMP101 uses the procedural paradigm
- Computational Thinking (‘problem solving’) is needed
 - Produce a good program design that solves the problem
 - Study the problem from the program specification and design an algorithm
 - a set of step-by-step instructions to be coded in any programming language
 - Pseudocode, Structured English, Flow Charts, Decision Tables, Venn diagrams etc

Algorithms

- Computer science studies solving of problems
- Solving of problems is described using algorithms
- Algorithms helps us understand the problem
- An algorithm is written 'language independent'
 - It does not reference any specific programming language
 - Different algorithms may solve the same problem
 - We try to use the best one that delivers the result

Algorithms

- Algorithms can be classified
 - Well-defined algorithm - steps are precise and understandable
 - Finite algorithm - must eventually solve the problem
 - Effective algorithm - solves all cases in the desired output
 - Flexible algorithm - handles a wide range of data inputs
- An algorithm can be 'encoded' in any programming language for execution by a computer
 - Using the design algorithm we write ('code') a program to run ('execute') on a computer
 - A program script ('code') describes a solution in terms of the data
 - a step-by-step list of instructions that represent the algorithm as a computer language



COMP101

Algorithms

- Algorithms describe a problem using ‘structured approaches’

1. ‘Divide and Conquer’

Structured English: No standard, is ambiguous, can cause confusion

Pseudocode: No standard but tighter than Structured English so more accurate

Flowcharts: Have a standard with different standards for different applications

Decision tables: Have a standard to compute an action depending on a condition
etc

2. Top-down Design vs Bottom-up Design

Decomposition/refinement: Can provide a view of a system as its component parts

Can use ‘black box’ approaches – the actual detail inside the box is not known, only the i/o

Algorithms

- Use a series of well-defined steps to solve the problem
- Start with a high level design: INPUT – PROCESS – OUTPUT
- Break this down to lower levels using structured approaches
- Is there something similar that has been applied to a similar problem?
- Eliminate non-workable solutions

From Algorithm into Program

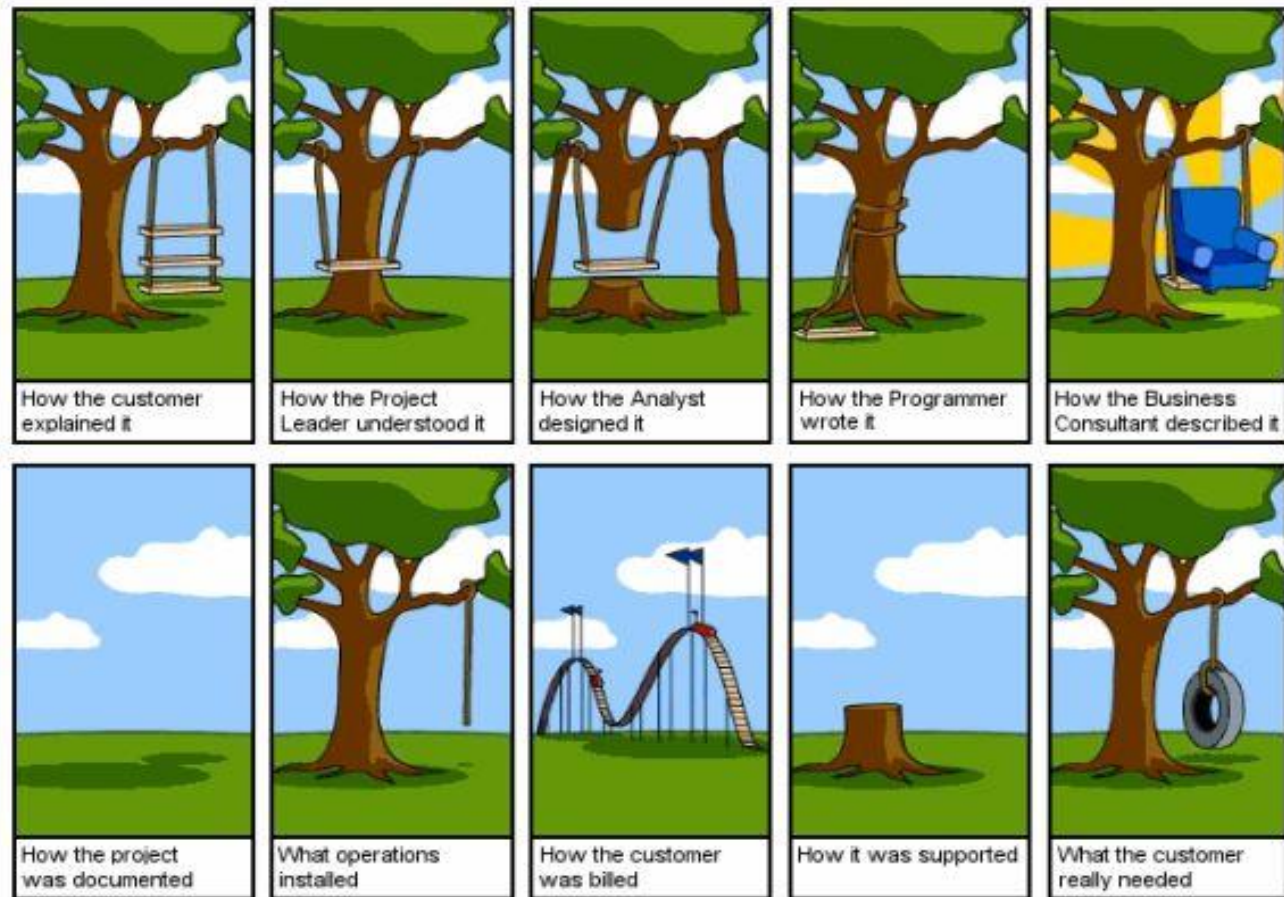
- We turn an algorithm into a program using ‘control constructs’, ‘data types’ and ‘operations’
- Control Constructs (three of them common to all programming languages)
 - **Sequence:** ‘sequence statements’ every line of code in the program will execute in the order written
 - Program WILL work only when each executes correctly – they must all execute
 - **Selection:** ‘conditional statements’. Not all lines execute
 - Program WILL work even if some lines do not execute
 - **Iteration:** ‘loop statements’. Some code can iterate many times, some may not
 - The program WILL work if these lines are iterated or not
- Data types (several types depending on language used)
 - Primitive types – **integer, string, floating point** etc (low-level built-in to the language to act as building blocks)
- Operations (many types across all languages)
 - Definitions of what can be done to the data, e.g. **addition, subtraction, multiplication, division** etc

Problem Solving

Complex problems can be difficult to understand when they get larger.
If we don't understand the problem we can code the wrong solution.
Often a problem is specified in 'natural language' with ambiguities

- We need a 'Problem-Solving' approach of well-defined steps
 - Pseudocode, Structured English, Flowcharts, Decision tables/Trees etc
- From the defined steps we get an algorithm to code the solution
 - Finite and effective – it works and gives the correct answer

Good problem solving avoids mistakes



Problem Statement:

Draft #1 (the 'rough statement')

“Calculate cost of electricity used”

- We can't program this – it lacks any detail
- 'refine' the problem statement: Start with Input – Process – Output

Draft #2 (refinement of draft#1 – address input-process-output)

“Take a meter reading to calculate electric used and produce a bill”

- Problem statements use natural language – can lead to error
 - Input – what data units?
 - Process – which calculation?
 - Output – what information?

Problem Statement:

refinements draft#3 and #4

- Draft#3 - refinement of **input**

“For a customer based on their power used as a meter reading given in kilowatthours (kWh), calculate electric used and produce a bill”

- Draft#4 - refinement of **process**

“For a customer based on their power used as a meter reading given in kilowatt hours (kWh), calculate total electric usage used as the meter reading multiplied by the cost of one kilowatt hour, plus the standing charge and produce a bill showing total electricity used”

Problem Statement: refinements draft#5

- Draft#5 - refinement of output

“For a customer based on their power used as a meter reading given in kilowatt hours (kWh), calculate total electric usage used as the meter reading multiplied by the cost of one kilowatt hour, plus the standing charge and produce a bill showing total electricity used, standing charge and total amount to pay”

Problem Specification

- Rough Statement supplied by client

“Calculate cost of electricity used”

- Problem Specification after refinement

“For a customer based on their power used as a meter reading given in kilowatt hours (kWh), calculate total electric usage used as the meter reading multiplied by the cost of one kilowatt hour, plus the standing charge and produce a bill showing total electricity used, standing charge and total amount to pay”

Problem Specification into Design

- We have enough detail to design a solution
 - There may be further refinements but we need to be aware some of these are cosmetic and others can be added at code-time
 - Customer name, address etc does not have an impact on the design
 - Don't add refinements that do not contribute to the fundamental design
- We take the Problem Specification and apply design techniques to it
 - Gives us an algorithm from which we can program a solution
 - Several design techniques exist: 'top-down design', 'stepwise refinement' etc
 - On paper these materialise as:
 - Structured English, pseudocode, flowcharts, decision tables etc